

AI-POWERED INTRUSION DETECTION & CYBERSECURITY MONITORING SYSTEM

Comprehensive Research Synthesis, Comparative Model Review, Technical Corrections & Master Implementation Blueprint

Target System: Sovereign AI-Driven NIDS / HIDS / SIEM / SOAR	License Constraint: 100% Free & Open-Source Software (FOSS)
Evaluated Models: Gemini 3.7, Qwen 3.8 Max, GPT-5.6 Luna, Claude Sonnet 5	Release State: August 2026 Production Master Specification

1. Executive Summary & Research Synthesis

Modern cybersecurity defense requires a paradigm shift from rigid signature matching to dynamic, multi-modal artificial intelligence. Sophisticated adversaries operate with polymorphic evasion, encrypted C2 beacons (TLS 1.3, QUIC), and multi-stage lateral movement campaigns that bypass traditional boundary firewalls and standard intrusion detection systems. Constructing a high-performance, cost-free, and sovereign security operations platform requires orchestrating proven open-source telemetry collectors, high-throughput streaming pipelines, in-process machine learning inference, and human-gated active response.

Across previous research iterations (contributed by **Gemini, Qwen 3.8 Max, GPT-5.6 Luna, and Claude Sonnet 5**), consensus was established around a core layered stack: **Suricata & Zeek** for network telemetry, **Wazuh & Sysmon** for endpoint detection, **Kafka/Flink** for data streaming, and **XGBoost/LightGBM** for tabular flow classification. However, earlier iterations contained critical latency blind spots, encrypted traffic oversights, dataset artifacts, and hardware sizing failures. This document synthesizes all research, resolves all discovered flaws, and delivers a production-grade master implementation plan.

2. Synthesis of Evaluated Multi-Model Research Materials

Each previous model contributed distinct architectural layers and iterative refinements:

- **Gemini Research Baseline:** Established the layered reference architecture (Zeek, Suricata, Wazuh, Sysmon, OpenVAS, MISP, Triton Server, TheHive, Keep, and Flink). Highlighted public benchmark datasets (UNSW-NB15, CIC-IDS2017/2018, TON_IoT, NSL-KDD), Explainable AI (SHAP/LIME), and Federated Learning (Flower/FATE). Flagged NVIDIA Triton CVE vulnerabilities.
- **Qwen 3.8 Technical Plan:** Introduced critical engineering simplifications: replacing heavy external gRPC round-trips to Triton with **in-process ONNX Runtime** inference for tabular flow scoring; separating Suricata (signature/IPS) from Zeek (metadata); establishing tabular tree algorithms (XGBoost/LightGBM) over transformer models for structured NetFlow telemetry; and adding a preliminary directory layout.
- **GPT-5.6 Luna & Claude Sonnet 5 Revisions:** Identified licensing and cloud drift: TheHive v5+ transitioned to commercial/restricted licensing (prompting the switch to DFIR-IRIS); Oracle Cloud cut Always-Free ARM allocation to 2 OCPU / 12GB RAM; highlighted that TON_IoT is restricted to non-commercial academic use; decoupled SHAP computation from LLMs (enforcing model-derived SHAP); and proposed shadow-mode scoring before active blocking.

3. Independent Deep-Dive Cybersecurity & AI Research (Gemini 3.7)

An independent technical analysis reveals several indispensable architectural components necessary for real-world enterprise defense:

3.1 Dual-Window Inspection: Eliminating the 'End-of-Flow' Real-Time Paradox

Network analysis engines such as Zeek emit connection records (conn.log) only upon **TCP connection termination (FIN/RST) or inactivity timeout (typically 5–15 minutes)**. Relying exclusively on conn.log for real-time intrusion prevention creates a fatal flaw: active attacks (such as data exfiltration or reverse shells) complete long before the model receives telemetry.

Engineering Solution: Implement a **Dual-Window Pipeline**. An *Early-Packet Micro-Window* inspects the first 10–20 packets of each new flow to compute line-rate features (packet lengths, inter-arrival times, TCP window dynamics, TLS ClientHello). An in-process ONNX model scores this micro-window within **< 2 milliseconds** for instant containment. Full-session logs are reserved for longitudinal behavioral baselining, beaconing detection, and forensic graphing.

3.2 Encrypted Traffic Analysis (ETA) & JA4+ Network Fingerprinting

With >90% of traffic encrypted via TLS 1.3 and QUIC (HTTP/3 over UDP), Deep Packet Inspection (DPI) looking for plaintext signatures is largely obsolete. Modern AI detection must extract side-channel and handshake features:

- **JA4+ Fingerprint Hashing:** JA4 (TLS client cipher suites, extensions, ALPN), JA4S (server responses), JA4H (HTTP header hashing), and JA4L (latency/light distance).
- **Sequence of Packet Length and Time (SPLT):** Vectorizing the byte sequence and inter-arrival timing of initial packets to classify traffic behavior without payload decryption.
- **Entropy & Handshake Analytics:** Measuring packet payload entropy, SNI frequency distribution, and certificate chain features to reliably detect C2 frameworks (Cobalt Strike, Sliver, Mythic).

3.3 Multi-Stage Correlation via Temporal Graph Neural Networks (GNNs)

Tabular models evaluate each network flow in isolation. Advanced Persistent Threats (APTs) execute multi-stage campaigns across different hosts (Reconnaissance → Lateral Movement → Privilege Escalation → Staging → Exfiltration). Individual flows often appear benign when viewed out of context.

Architectural Upgrade: Construct a **Temporal Interaction Graph (TIG)** powered by **E-GraphSAGE** or **Dynamic Graph Neural Networks**. Host endpoints, user accounts, and external IPs form graph nodes, while authentication events and network connections form timestamped directed edges. Graph embeddings uncover distributed port scans, Kerberoasting, and lateral movement across subnets.

4. Critical Comparative Matrix & Required Corrections

The following matrix summarizes the comparative discrepancies and necessary engineering corrections across all reviewed research artifacts:

Evaluation Dimension	Prior Synthesized Proposals	Independent Technical Assessment	Impact & Correction Required
Network Latency & Flow Timing	Zeek conn.log forwarded to Kafka/ONNX for sub-second blocking.	Zeek emits conn.log on connection close/timeout (5-15 min delay).	Critical Flaw: Must add Early-Flow micro-windowing (first 10-20 pkts) for real-time blocking.
Encrypted Traffic Visibility	Generic DPI & basic packet headers (bytes, duration, ports).	90%+ traffic is encrypted (TLS 1.3/QUIC); DPI yields zero signal.	Major Omission: Must ingest JA4+ fingerprints & SPLT vectors.
ML Model Scope & Graph APT	Strictly tabular tree models (XGBoost/LightGBM) & Autoencoders.	Tabular models are blind to multi-hop lateral movement & APT chains.	Enhancement: Add Temporal Graph Neural Networks (TGN / E-GraphSAGE).
Benchmark Datasets	UNSW-NB15, CIC-IDS2017, NSL-KDD, TON_IoT.	NSL-KDD is from 1999; raw CIC-IDS2017 has severe socket/TTL leakage.	Correction: Standardize on NetFlow v9 (NF-UQ-NIDS-v2) & USTC-TFC2016.
SOAR & Orchestration	DFIR-IRIS for case management; Keep for alert workflows.	DFIR-IRIS is a case tracker, not a SOAR engine. Keep is enterprise-gated.	Tooling Upgrade: Introduce Shuffle SOAR (100% open-source workflow automation).
Free Cloud / RAM Reality	Run Wazuh Indexer + Manager + Zeek + Suricata on 12GB Oracle Cloud.	OpenSearch JVM (6GB) + Zeek (3GB) + Wazuh (2GB) will trigger Linux OOM panics.	Sizing Fix: Provide Lean Profile (Vector + ClickHouse) for <= 16GB RAM.
AI / LLM Defense Threat Surface	Brief mention of prompt injection via log fields.	Attackers can inject control prompts via DNS queries/User-Agents.	Security Mandate: Dual-tier LLM isolation, strict Pydantic JSON validation.

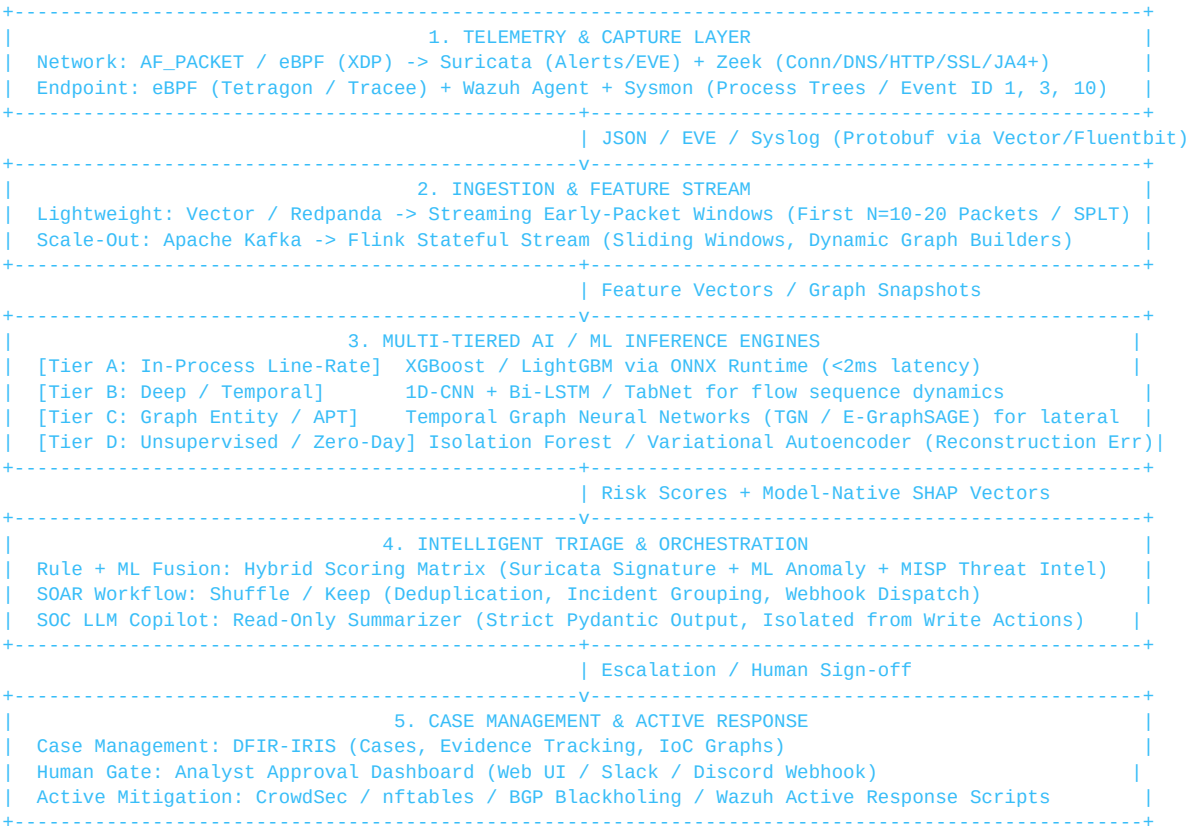
Summary of Definitive Corrections Applied in Master Plan:

- **1. Latency & Dual-Window Pipeline:** Early-packet feature extraction (first 10–20 packets) enables sub-2ms line-rate inference via in-process ONNX Runtime, eliminating the connection-close latency paradox.
- **2. Encrypted Traffic Handling:** Standardized feature vectors to incorporate JA4/JA4S fingerprints, TLS handshake telemetry, and Sequence of Packet Length and Time (SPLT).
- **3. Dataset Standard Overhaul:** Replaced flawed raw benchmarks with NetFlow v9 standardized benchmarks (**NF-UQ-NIDS-v2**, **NF-UNSW-NB15-v2**) and encrypted malware sets (**USTC-TFC2016**).
- **4. Memory Sizing & Lean Architecture:** Introduced a low-memory profile using **Vector + ClickHouse** for environments under 16GB RAM to prevent out-of-memory kernel panics.
- **5. Dedicated SOAR Automation:** Integrated **Shuffle SOAR** alongside DFIR-IRIS to automate enrichment (MISP/VirusTotal) and execute human-gated active containment (CrowdSec/Wazuh).
- **6. SOC Copilot Defense:** Enforced strict Pydantic JSON schema validation and isolated LLMs to read-only summarization, neutralizing prompt injection via log fields.

5. Master Production Implementation Plan

This master plan provides a complete, buildable blueprint incorporating all corrections, hardware profiles, operational workflows, and security guardrails.

5.1 End-to-End System Reference Architecture



5.2 Deployment Profiles & Hardware Sizing

Component Layer	Profile A: Lightweight / Lab (<= 16GB RAM)	Profile B: Distributed Enterprise (32GB+ RAM)
Network Capture	Suricata (AF_PACKET) + Zeek (Light scripts)	Suricata (DPDK/eBPF) + Zeek (Full JA4+ Cluster)
Host Monitoring	Wazuh Agent + Sysmon (Windows) / auditd (Linux)	Wazuh Agent + Sysmon + eBPF Tetragon/Tracee
Log Ingestion	Vector (Rust-based, < 50MB RAM)	Vector + Apache Kafka Message Cluster
Stream & ML Engine	In-process Python ONNX Consumer (sub-2ms)	Apache Flink Stateful Stream + ONNX C++ Plugin
Storage & DB	ClickHouse Columnar Storage (< 2GB RAM)	ClickHouse (Cold/Hot Flows) + OpenSearch Cluster
SOAR & Cases	Shuffle SOAR (Community) + DFIR-IRIS	Shuffle SOAR + DFIR-IRIS + Cortex + MISP Cluster
Active Response	CrowdSec Bouncer + Local iptables/nftables	CrowdSec + BGP Blackholing + Wazuh Active Resp.

5.3 Phased 8-Stage Execution Roadmap

- **Phase 0: Sizing & Infrastructure Setup (Day 1):** Select Profile A (Local VM / Cloud) or Profile B (Distributed Cluster). Stand up base Docker environment and network tap/SPAN port.
- **Phase 1: Telemetry Baseline (Week 1):** Deploy Wazuh Manager and agents across endpoints. Deploy Suricata with ET Open rules and Zeek on the network SPAN port. Validate live JSON/EVE telemetry.
- **Phase 2: Ingestion & Storage Pipeline (Week 2):** Deploy Vector log shipper to parse Suricata EVE and Zeek JSON logs directly into ClickHouse. Verify ingest throughput and storage compression.
- **Phase 3: Feature Engineering & Early-Flow Pipeline (Weeks 3–4):** Implement early-packet feature extractor (first 10–20 packets + JA4+ TLS parser). Generate standardized 43-feature NetFlow vectors.
- **Phase 4: ML Model Training & Offline Calibration (Weeks 5–6):** Train LightGBM/XGBoost classifiers and Isolation Forest autoencoders on NF-UQ-NIDS-v2 and USTC-TFC2016. Export to ONNX via skl2onnx. Enforce Platt scaling calibration and PR-AUC validation.
- **Phase 5: Shadow-Mode Stream Scoring (Weeks 7–8):** Deploy in-process ONNX scorer alongside Vector. Run for 2 weeks in shadow mode (generating risk scores and TreeSHAP feature explanations without modifying alert severity). Audit false-positive rate per host-day.
- **Phase 6: Case Management & SOAR Workflows (Weeks 9–10):** Deploy DFIR-IRIS and Shuffle SOAR. Configure playbooks to ingest high-confidence ML alerts, enrich with MISP IoCs, and dispatch approval requests via webhook.
- **Phase 7: SOC Copilot & Guardrail Integration (Week 11):** Deploy local read-only LLM (Ollama/vLLM) to translate model-derived TreeSHAP feature vectors into concise analyst summaries. Enforce strict Pydantic JSON output validation.
- **Phase 8: Adversarial Emulation & Verification (Week 12, Ongoing):** Execute automated adversary emulation (Nmap scans, Hydra brute-force, Sliver C2 beacons). Verify full pipeline: capture -> early-flow ONNX score -> Shuffle triage -> human gate -> CrowdSec containment.

5.4 Production Codebase & Directory Structure

```
sovereign-ai-ids-siem/
|-- docker-compose.yml           # Master container orchestrator
|-- docker-compose.override.yml  # Sizing overrides (Profile A vs Profile B)
|-- config/
|   |-- suricata/                # suricata.yaml, custom rules, lua early-flow scripts
|   |-- zeek/                   # local.zeek, ja4/ extension packages
|   |-- wazuh/                  # ossec.conf, custom decoders, custom rules
|   |-- vector/                 # vector.yaml (sub-50MB RAM log router)
|   |-- shuffle/                # Shuffle SOAR workflows & webhook configs
|   |-- clickhouse/             # Columnar schema definitions (network_flows, endpoint_logs)
|   |-- grafana/                # Pre-provisioned security dashboards & alerts
|-- ml_pipeline/
|   |-- datasets/                # NF-UQ-NIDS-v2, USTC-TFC2016, raw pcaps
|   |-- features/
|       |-- extractor_early_flow.py # First 10-20 packet window feature generator
|       |-- ja4_extractor.py       # JA4/JA4S TLS fingerprint parser
|       |-- flow_aggregator.py     # Standardized 43-feature NetFlow v9 generator
|   |-- training/
|       |-- train_tabular_trees.py # XGBoost / LightGBM + Optuna hyperparam tuning
|       |-- train_autoencoder.py   # PyTorch Unsupervised Reconstruction Model
|       |-- export_onnx.py         # skl2onnx & torch.onnx converters
|   |-- eval/
|       |-- evaluate_temporal.py   # Time-series split, PR-AUC, host-day FP calculation
|       |-- calibration_curves.py  # Brier score & Platt scaling verification
|       |-- models/               # Serialized .onnx model artifacts + SHA256 signatures
|-- inference_engine/
|   |-- stream_scorer.py          # In-process Python/C++ ONNX Runtime consumer
|   |-- tree_shap_explainer.py    # C-optimized TreeSHAP engine for flow vectors
|   |-- test_stream.py            # tcpreplay performance & latency harness
|-- soc_copilot/
|   |-- summarizer.py             # Read-only Ollama/vLLM client
|   |-- schemas.py                # Pydantic schema validation for copilot outputs
|   |-- prompt_sanitizer.py       # Indirect prompt-injection sanitization filters
|-- dfir_iris/                   # DFIR-IRIS case management compose templates
`-- tests/
    |-- attack_scenarios/         # Automated adversary emulation (Nmap, Hydra, Sliver)
    |-- replay/                   # Malicious PCAP replayer for validation
```

5.5 Rigorous ML Evaluation Discipline & Security Guardrails

- **Zero-Data-Leakage Splitting:** Never use random train/test splits. Always split data chronologically or by network subnet. Fit encoders and scalers exclusively on the training partition.
- **Metric Hierarchy:** Discard raw classification accuracy on imbalanced network data. Optimize strictly for **Precision-Recall AUC (PR-AUC)** and track **False Positives per Host-Day (FP/host-day)**.
- **Strict Shadow Mode:** New models must operate in shadow mode for a minimum of 14 days of live network traffic before their risk score is permitted to influence alert escalation.
- **Mandatory Human Approval Gate:** All automated containment actions (CrowdSec firewall IP blocks, Wazuh host isolations) must be gated behind human approval webhooks during initial deployment.
- **Dual-Tier LLM Isolation:** The SOC Copilot LLM operates strictly in read-only mode with no write permissions to firewalls, databases, or case managers. All raw telemetry inputs are sanitized to prevent prompt injection.

End of Master Reference Report | Generated for Sovereign Cybersecurity Defense